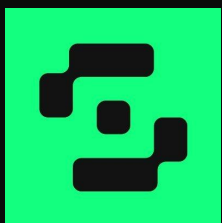




Security Assessment Final Report



Safe Individual Policies

September 2026

Prepared for Safe

Table of Contents

Project Summary	3
Project Scope	3
Project Overview	3
Protocol Overview	4
Assessment Methodology	5
Threat and Security Overview	6
Findings Summary	7
Severity Matrix	7
Detailed Findings	8
Informational Severity Issues	9
I-01: `getCoSigner` is not properly usable	9
I-02: MultiSend checks the transfer using an outdated Safe threshold	10
Disclaimer	12
About Certora	12

Project Summary

Project Scope

Project Name	Repository Link	Commit Hash	Platform
Safe Individual Policies	https://github.com/safe-research/policy-engine	Initial: 405ba1d Final: 8c30032	Solidity / EVM

Project Overview

This document describes the security review of **Safe Individual Policies**. The work was undertaken from **September 16th, 2026** to **September 21st, 2026**.

The following contract list is included in our scope:

contracts/policies/AllowedModulePolicy.sol
contracts/policies/AllowPolicy.sol
contracts/policies/DenyPolicy.sol
contracts/policies/CoSignerPolicy.sol
contracts/policies/ERC20ApprovePolicy.sol
contracts/policies/ERC20TransferPolicy.sol
contracts/policies/IncreasedThresholdPolicy.sol
contracts/policies/MultiSendPolicy.sol
contracts/policies/NativeTransferPolicy.sol
contracts/policies/SafenetPolicy.sol

The team performed a manual audit of all the files in scope. During the manual audit, the Certora team discovered bugs in the code, as listed on the following page.

Protocol Overview

The project adds a configurable policy layer to Safe wallets. It lets owners define what their wallet is allowed to do, alongside the usual requirements.

Individual policies are the building blocks of this system. Each policy contains a specific set of conditions for allowing a transaction, such as restricting recipients, requiring additional approval, or granting permission for a single use. Owners assign policies to particular actions and can configure a fallback policy for other transactions. If no policy permits an action, it is blocked.

The design separates the engine that selects and enforces policies from the rules implemented by each policy. This makes it possible to introduce new rules without changing the core engine. Policy changes go through a configurable delay once the guard is installed, giving owners time to review or cancel proposed changes.

Assessment Methodology

Our assessment approach combines design level analysis with a deep review of the implementation to ensure that a protocol is secure, economically sound, and behaves as intended under realistic conditions.

At the design level, we evaluate the architecture, the economic assumptions behind the protocol, and the safety properties that should hold independently of a specific chain or environment. This process includes reviewing internal and cross protocol interactions, state transition flows, trust boundaries, and any mechanism that could be exploited to extract value, deny service, or alter core system behavior. At this stage, a focused threat modelling exercise helps identify key attack surfaces and adversarial capabilities relevant to the system. Design level issues often relate to incentive structures, governance implications, or systemic behavior that emerges under adversarial conditions.

Implementation analysis focuses on the concrete behavior of the code within the execution model of the target chain. This involves reviewing the correctness of logic, access control, state handling, arithmetic behavior, and the nuanced behaviors of the chain environment. Familiar classes of vulnerabilities such as reentrancy conditions, faulty permission checks, precision issues, or unsafe assumptions often surface at this layer. These findings require context aware reasoning that takes into account both the code and the architectural intent.

To support this analysis, the codebase is examined through repeated manual passes and supplemented by automated tools when appropriate. High-risk logic areas receive deeper scrutiny, invariants are validated against both design intent and actual implementation, and potential vulnerability leads are thoroughly investigated. Automated techniques such as static analysis, fuzzing, or symbolic execution may be used to complement manual review and provide additional insight.

Collaboration with the development team plays an important role throughout the audit. This helps confirm expected behaviors, clarify design assumptions, and ensure an accurate understanding of the protocol's intended operation. All findings are documented with clear reasoning, reproducible examples, and actionable recommendations. A follow up review is conducted to validate the applied fixes and verify that no regressions or secondary issues have been introduced.

Threat and Security Overview

The system adds policy enforcement to Safe wallets, covering both owner transactions and module execution. Each action is checked against an exact policy or an operation-specific fallback, with unmatched actions denied. Owners manage configuration through a delay, while policies can require additional signatures, restrict recipients or spenders, or grant single-use permissions. Funds remain in the Safe.

The main security assumptions are that both guards are installed, the Safe version is compatible, and configured policies and delegatecall targets are trusted. Module identity comes from the guard; caller-supplied context must be independently authenticated.

Manual inspection focused on transaction routing, configuration, individual policy checks, batch handling, signature validation, and Safenet key management. Attack vectors considered included compromised owners or modules, forged context, reentrancy, replay, permissive fallbacks, guard removal, malformed calldata, and mismatches between checked and executed actions. Dependency boundaries include Safe, MultiSend, contract signers, token contracts, and Safenet validators. Economic exposure comes primarily from unauthorized transfers and approvals: recipient restrictions do not impose spending limits, and approved spenders can later withdraw tokens without another Safe transaction.

The standard test suite passed all 175 tests, covering policy selection, owner and module paths, configuration timing, allowlists, single-use grants, signatures, batches, reentrancy, Safe isolation, and execution failures.

Findings Summary

The table below summarizes the findings of the review, including type and severity details.

Severity	Discovered	Confirmed	Fixed
Critical	–	–	–
High	–	–	–
Medium	–	–	–
Low	–	–	–
Informational	2	2	2
Total	2	2	2

Severity Matrix

Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Low	Low	Medium
		Low	Medium	High
Likelihood				

Detailed Findings

ID	Title	Severity	Status
I-01	`getCoSigner` is not properly usable	Informational	Fixed
I-02	MultiSend checks the transfer using an outdated Safe threshold	Informational	Fixed

Informational Severity Issues

I-01: `getCoSigner` is not properly usable

Files:

contracts/policies/CoSignerPolicy.sol

Description:

`getCoSigner()` works as follows:

```
function getCoSigner(address safe, AccessSelector.T access) external view returns
(address cosigner) {
    cosigner = $cosigners[msg.sender][safe][access];
}
```

The cosigners mapping is keyed by policy engine, safe and access selector. For the function to properly return a cosigner, the policy engine must be able to call it due to the `msg.sender` usage. However, there is no way to call the function through the policy engine, thus the only way to call it is in cases where the `msg.sender` is not actually a policy engine, making the function not usable (unless, for example, queried off-chain using `eth_call` through an RPC to mock the caller) as it does not return an actual cosigner.

Recommendations:

Consider changing the `msg.sender` to instead be an input to the function. Alternatively, add a function in the policy engine that can call `getCoSigner()`.

Customer Response:

Fixed in [9f62bc2](#).

Fix Review:

Confirmed.

I-02: MultiSend checks the transfer using an outdated Safe threshold

Files:

contracts/policies/MultiSendPolicy.sol

Description:

MultiSendPolicy checks every transaction in a batch before executing any of them. Because of this, **IncreasedThresholdPolicy** can calculate its signature requirement using an old threshold even when an earlier transaction in the same batch will increase that threshold.

Consider the following scenario ->

The Safe has three owners Alice, Bob, and Charlie and currently requires one owner signature. **IncreasedThresholdPolicy** requires one additional signature, so a transfer currently needs two signatures.

A batch contains:

1. Change the Safe threshold from 1 to 2.
2. Transfer 100 tokens to Bob.

Before executing the batch both transactions are checked. At this point the threshold is still 1, so the transfer is approved using only Alice and Bob's two signatures.

The batch then raises the threshold to 2 and executes the transfer without checking it again. Under the new threshold, the transfer should have required three signatures, including Charlie's.

As a result a transaction can execute with fewer signatures than the policy would require based on the Safe's state at the moment of execution.

Recommendations:

Consider acknowledging and documenting the case as a design limitation for user awareness.

Customer Response:

Documented in [8c30032](#).

Fix Review:



Confirmed.

Disclaimer

Even though we hope this information is helpful, we provide no warranty of any kind, explicit or implied. The contents of this report should not be construed as a complete guarantee that the contract is secure in all dimensions. In no event shall Certora or any of its employees be liable for any claim, damages, or other liability, whether in an action of contract, tort, or otherwise, arising from, out of, or in connection with the results reported here.

About Certora

Certora is a Web3 security company that provides industry-leading formal verification tools and smart contract audits. Certora's flagship security product, Certora Prover, is a unique SaaS product that automatically locates even the most rare & hard-to-find bugs on your smart contracts or mathematically proves their absence. The Certora Prover plugs into your standard deployment pipeline and is widely used by developers and security researchers during audits and bug bounties.

Certora also provides architectural design reviews, where researchers analyze system design and trust boundaries early to identify structural risks and reduce complexity before implementation.

Certora conducts off-chain audits covering backend services, APIs, frontend, and mobile applications, focusing on vulnerabilities in authentication, data handling, key management, and wallet-related workflows.

Certora also offers Penetration Testing and Red Teaming to simulate real-world attacks and uncover exploitable weaknesses across infrastructure, applications, and business logic.

In addition, Certora provides operational security (OpSec) assessments, reviewing key management, access controls, and operational processes to strengthen overall security posture.

Finally, Certora delivers ongoing monitoring and incident response, enabling continuous threat detection and prevention, alert triage, and coordinated response to active incidents.